ConsensusScope: Safety Routing for Generative AI English Essay Feedback

You Tingrui[1], Li Jimei[2*], Zhang Na[2], Xiong Ling[2], Zhang Xiaodan[1], Xiao Shurou[1], and Yang Yue[1]
[1]Hainan International College, Beijing Language and Culture University
[2]School of Information Science, Beijing Language and Culture University
yyting2006@163.com, Ljm@blcu.edu.cn, zn17333099816@163.com, lingdonghua99@163.com
{202511681449, 202511681363, 202511681400}@stu.blcu.edu.cn

Abstract

Generative AI can reduce the workload of English essay feedback, but fluent suggestions may still change student meaning, add unsupported content, or overcorrect a draft. We present ConsensusScope, an interactive system for safety routing of AI-generated English essay feedback. The system treats each feedback item, rather than an essay score or final rewrite, as the unit of control: it normalizes candidates, extracts deploy-time safety signals, constructs an auditable Feedback Safety Graph, and applies a pilot-calibrated logistic release policy to route each item to auto_accept, teacher_review, or needs_more_evidence. A Streamlit/FastAPI demo supports single-essay review, batch review, teacher queueing, graph inspection, and report export. We evaluate routing with a stress suite, a public learner-corpus benchmark from JFLEG, CoNLL-2014, FCE, and W&I+LOCNESS, and a 30-item two-teacher pilot. Across 349,782 generated feedback candidates, the default policy auto-routes 32.9% while sending all constructed incorrect or unsafe candidates to review. The teacher pilot shows 0.857 auto-accept precision against teacher-safe items and 0.694 within-one-point teacher agreement. These results support graph-backed safety routing, not automatic essay scoring or teacher replacement.

1 Introduction

Generative AI is becoming a plausible assistant for English essay feedback, a task with a long history in grammatical error correction (GEC) benchmarks (Ng et al., 2014; Bryant et al., 2019), learner-writing corpora (Yannakoudakis et al., 2011; Napoles et al., 2017), and automated writing support (Madnani et al., 2018). The practical appeal is clear: teachers can obtain grammar, vocabulary, organization, and argument comments at essay scale. The unresolved bottleneck is safe release. A fluent AI comment can be wrong after a modal verb, shift the student's claim, add unsupported evidence, or give advice that is too broad to show directly to learners. Reviewing every suggestion removes the promised workload reduction; releasing every suggestion risks misleading students.

ConsensusScope formulates this bottleneck as item-level safety routing. Given an anonymized ESL draft and AI feedback candidates, the system routes each item to auto_accept, teacher_review, or needs_more_evidence. The decision unit is a concrete feedback item tied to a target span, local context, suggestion, rationale, and issue type, rather than an essay score or final rewrite.[1]

The core mechanism is Feedback Safety Graph Routing. ConsensusScope links the target span, AI suggestion, evidence status, active safety dimensions, and final route in an auditable graph. Deploy-time signals include meaning-preservation warnings, content-grounding risks, local-edit scope, tone and specificity, model agreement, and parse failures. The system therefore exposes why an item is released, reviewed, or flagged as needing more evidence.

We evaluate this middle layer with stress checks, a public learner-corpus benchmark over 349,782 generated feedback candidates, and a 30-item two-teacher diagnostic pilot. The evidence supports a narrow claim: ConsensusScope can auto-route low-risk feedback while keeping constructed unsafe items in the

[1]Code and hosted demo: https://github.com/yyting2006-ai/ConsensusScope and https://demo.consensusscope.cn/.

review queue. It does not replace teachers or measure classroom learning gains. The demo contributes a safety-routing formulation, a Feedback Safety Graph Routing algorithm with pilot-calibrated release policy, a usable Streamlit/FastAPI review system, and a reproducible evaluation package for teacher-supervised AI feedback.

2 Related Work

Learner writing and GEC. GEC benchmarks have provided standardized correction tasks and learner-writing data, including CoNLL-2014 (Ng et al., 2014), BEA-2019 W&I+LOCNESS (Bryant et al., 2019), FCE (Yannakoudakis et al., 2011), and JFLEG (Napoles et al., 2017). These datasets primarily evaluate correction quality. ConsensusScope uses their correction annotations only as offline evidence for a downstream routing question: whether risky or incorrect feedback candidates remain in a review queue.

Automated writing feedback. Automated writing systems can provide revision support, but classroom use depends on how learners and teachers interpret the feedback (Madnani et al., 2018; Ranalli, 2018). ConsensusScope follows this caution by avoiding a teacher-replacement claim and making AI feedback inspectable before it becomes student-facing.

LLM reliability and adjudication. Multi-output and judge-based methods can help select answers or evaluate generations (Wang et al., 2023; Zheng et al., 2023; Liu et al., 2023). However, consensus or judge confidence is not equivalent to educational safety. Our demo treats model agreement as one observable signal alongside issue type, evidence status, meaning-change warnings, unsupported-claim cues, tone cues, and parse errors.

Unlike GEC tools, essay scorers, or generic LLM-as-judge workflows, ConsensusScope routes each feedback item before it reaches a student and exposes graph evidence behind release or review decisions.

3 Feedback Safety Graph Routing

ConsensusScope is organized around an item-level routing algorithm embedded in a teacher-facing review system. A teacher can paste a single anonymized ESL draft or upload a batch CSV. The system accepts generated feedback candidates in a unified schema, screens each item with the Feedback Safety Graph Router, and displays it as either a low-risk local edit or a teacher-review item.

Unified feedback schema. Every feedback candidate is represented with a stable item-level schema: feedback_item_id, essay_id, target span, surrounding context, AI suggestion, rationale, model source, predicted issue type, and optional model agreement. This schema lets deterministic demo reviewers and live OpenAI-compatible providers feed the same routing and reporting components.

Problem definition. Let $f = (x, c, u, t, e, a)$ denote a feedback item with target span x, local context c, AI suggestion u, predicted issue type t, evidence status e, and optional agreement signal a. The router estimates $R(f) = (r, p, s, G)$, where r is a release route, p is a calibrated probability that teacher review is needed, $s \in [0, 1]$ is a bounded rule-derived risk score, and G is an auditable graph connecting the observable signals to the route. The deploy-time objective is conservative: release only items whose observable signals indicate low-risk local feedback, while sending ambiguous, unsupported, meaning-changing, or unsafe items to teacher review.

Algorithm. The router extracts deploy-time signals such as issue-type risk, parse failure, evidence conflict, model agreement, meaning-change cues, unsupported claims, whole-draft rewriting, tone warnings, teacher-dependent wording, and broad target spans. These signals produce human-readable risk reasons and a bounded risk score $s \in [0, 1]$, projected into six graph dimensions: local edit, meaning preservation, content grounding, pedagogical tone, feedback specificity, and model agreement. A lightweight logistic calibration layer, fit on the two-teacher diagnostic pilot, maps these graph features to a review probability p. The release/review cutoff is selected on the pilot to preserve review-needed recall while maximizing auto-release among teacher-safe items. High-severity graph signals can still override the learned cutoff and force teacher review. Teacher ratings and public-corpus gold correc-

System type	Main object	Typical output	ConsensusScope distinction
GEC / revision tools	Sentence edit	Corrected text	Audits whether feedback should be student-facing
Essay scoring systems	Whole essay	Score or rubric label	Avoids grading and routes item-level AI feedback
LLM-as-judge workflows	Model output	Preference or quality score	Uses graph-backed deploy-time safety signals
Feedback dashboards	Review records	Tables or filters	Connects target span, suggestion, risk, route, and teacher action

Table 1: ConsensusScope is positioned as feedback safety routing rather than automatic correction, essay scoring, or generic model judging.

tions are reserved for offline calibration and evaluation.

Graph construction. For each feedback item, ConsensusScope constructs a compact directed graph with the core path target span -> AI suggestion -> active safety dimension -> routing decision. Graph nodes represent the student target span, surrounding context, AI suggestion, optional rationale, predicted issue type, evidence signal, activated safety dimensions, and route. Edges capture relations such as contains, is_revised_by, activates, and justifies_route. The exported route includes graph dimensions, active signals, a path summary, and JSON node/edge records.

Routing outputs. For each feedback item, the system outputs a risk level, recommended action, risk reasons, risk score, graph path, node/edge records, and a short explanation. Low-risk grammar, spelling, punctuation, and vocabulary edits can be routed to auto_accept; medium- and high-risk items are routed to teacher review, more-evidence, or reject states.

4 Functionality

The Streamlit/FastAPI demo exposes the router as a teacher-facing workflow: single-essay review, batch review, AI feedback comparison, teacher queue, evaluation, and report export. Users inspect routed items, open graph paths, prioritize review queues, and save teacher decisions through the backend.

The intended users are English writing teachers, educational NLP developers, and researchers studying human-in-the-loop feedback. Teachers use the system to separate low-risk local edits from suggestions that still require professional judgment. Developers use normalized records to debug generative-feedback pipelines, and researchers can compare routing policies, risk signals, and teacher-aligned diagnostics.

The workflow is designed around teacher control rather than automatic release. In single-essay mode, a teacher can inspect how local spans, AI suggestions, and graph dimensions produce a route. In batch mode, the teacher queue prioritizes medium- and high-risk feedback items before student-facing reports are exported. This design makes the system useful even when the underlying feedback generator is replaced, because the routing layer only assumes a unified feedback schema.

Interfaces. The web interface is the primary demo surface for teachers. It exposes the essay view, feedback-item queue, graph explanation panel, batch upload, and student-report export. The FastAPI backend exposes the same routing functions for programmatic use, so developers can submit feedback items and retrieve the route, risk reasons, calibrated review probability, and graph record without using the Streamlit front end. The repository also includes command-line scripts for regenerating public-corpus benchmarks and calibration artifacts.

5 End-to-End Use Case

Consider a teacher reviewing a comparative-literature essay written by an English learner. A generative model may produce multiple suggestions: a safe local grammar edit, a vocabulary substitution that slightly changes the student's stance, an organization comment about paragraph order, and a content suggestion that introduces evidence not present in the essay. In a conventional AI-feedback interface, these suggestions are often displayed as a flat list, so the teacher must inspect every item before deciding what can be shown to the student.

ConsensusScope inserts a review-routing layer between generation and release. Each suggestion is converted to the unified feedback schema with its target span, local context, AI suggestion, rationale, issue type, and optional model agreement. The router then constructs a Feedback Safety Graph. A low-risk local grammar edit may activate the local-edit dimension and be routed to auto_accept. A vocabulary edit that changes the student's intended meaning activates meaning preservation and is routed to teacher_review. A content suggestion without support in the draft activates content grounding and is also routed to review. The teacher queue therefore focuses attention on items where professional judgment matters most.

The same workflow also supports batch review. When many essays are uploaded, the teacher can first inspect high-risk or evidence-poor feedback, then export a student-facing report containing only accepted or edited items. The exported audit record preserves the route, risk reasons, graph path, and teacher action, so later error analysis can identify which feedback types repeatedly require human intervention. This is the central application value of the system: it does not make AI feedback automatically correct, but it makes feedback release observable, reviewable, and safer for teacher-supervised use.

6 Evaluation

We evaluate the routing algorithm, not a full feedback generator or student learning outcome. The question is whether Feedback Safety Graph Routing separates safe local edits from incorrect, meaning-changing, unsupported, or unsafe feedback and whether its signals match teacher-facing diagnostics.

Algorithmic stress checks. The packaged demo contains 15 feedback items with expected routes and 16 stress-test cases for dangerous suggestions such as thesis reversal, whole-essay rewriting, unsupported factual claims, harsh tone, low agreement, and parse failures. The router matches the expected action and risk labels on all 31 items; this is an algorithm sanity check, not a classroom study.

Public learner-corpus benchmark. We construct an offline routing benchmark from public learner-correction corpora. For each original/corrected pair, we align tokens to create feedback-level gold edits, then generate one gold-derived correct candidate and risky distractors. The router sees only deploy-time feedback fields; gold labels are used afterward to check whether unsafe candidates were routed to review (Table 2).

Two-teacher diagnostic pilot. To test whether the review routes align with human judgments, we collected a small blind pilot from two English teachers over 30 anonymized AI feedback items. Each teacher rated correctness, meaning preservation, student readiness, usefulness, clarity, and direct-release suitability on 1–5 scales. The ratings are offline diagnostics: individual teacher labels are not read at deploy time, while the aggregate pilot is used to fit the lightweight calibration artifact. The pilot exposed borderline auto-release patterns: teacher-dependent wording, semantic drift in vocabulary edits, and wrong local grammar corrections after modal verbs. On the 30-item pilot, the final router sends 76.7% of items to review, reaches 1.000 recall for teacher-marked review-needed and unsafe items, and obtains 0.857 auto-accept precision against teacher-safe items (Table 3). Within-one-point teacher agreement is 0.694.

Case analysis. The pilot cases show the intended behavior: local phrase improvements can be auto-accepted, while thesis reversal, unsupported exam-score claims, and wrong modal-verb corrections are routed to review through meaning-preservation or content-grounding graph dimensions, without using teacher ratings at deploy time.

Review-routing utility. Table 4 summarizes the pooled policy results over 349,782 feedback candidates. The default policy accepts only low-risk items and routes medium/high-risk items to review. It auto-routes 32.9% of candidates while preserving all constructed incorrect or unsafe candidates in the review queue.

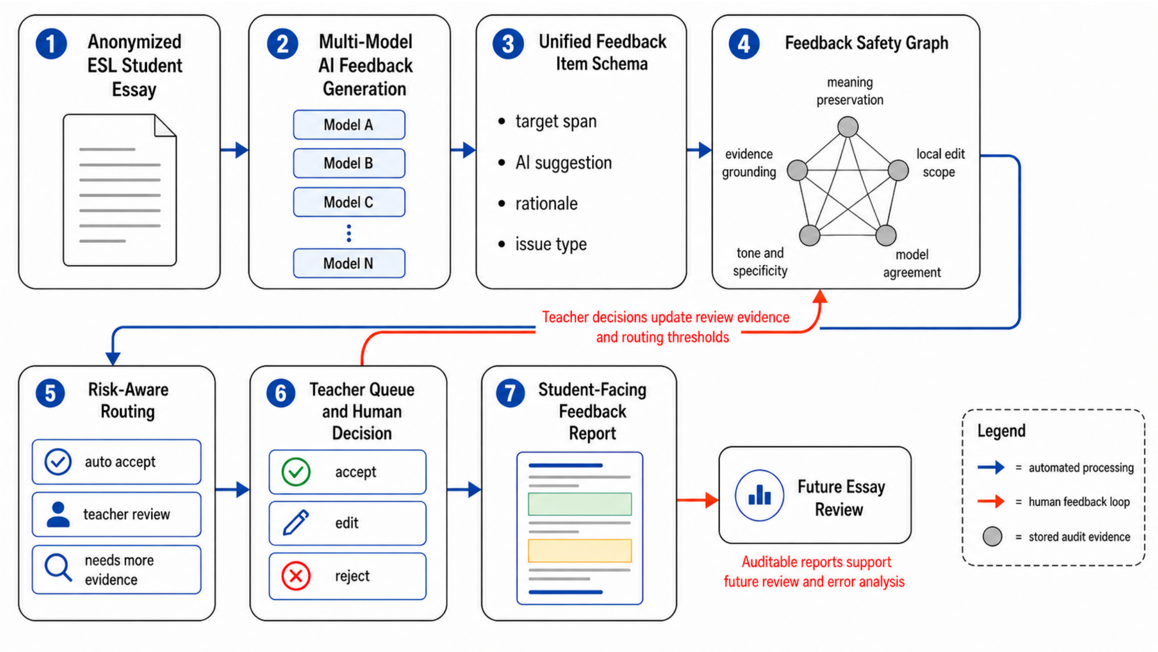

Figure 1: Feedback Safety Graph routing cycle in ConsensusScope. Blue arrows denote automated processing from anonymized essays and multi-model feedback to item-level routing decisions, while red arrows denote teacher feedback loops that update calibration evidence, review evidence, and future audit analysis.

Source	Records	Gold edits	Candidates	Auto share	Review share
JFLEG	1,501	3,938	11,587	0.320	0.680
CoNLL-2014	2,624	5,110	15,126	0.324	0.676
FCE	33,236	45,969	136,934	0.329	0.671
W&I+LOCNESS	38,692	62,384	186,135	0.329	0.671
Total	76,053	117,401	349,782	0.328	0.672

Table 2: Public learner-corpus routing benchmark.

Teacher-aligned diagnostic	Value
Feedback items	30
Rating rows	60
Auto share	0.233
Review share	0.767
Review-needed recall	1.000
Unsafe reviewed recall	1.000
Auto precision vs. teacher-safe	0.857
Within-one-point teacher agreement	0.694

Table 3: Two-teacher offline diagnostic pilot. Teacher ratings are used for offline calibration and diagnosis, not as per-item labels during deployment.

The 1.000 auto accuracy is a controlled diagnostic: correct candidates are derived from public gold corrections, and incorrect candidates are constructed risk distractors.

Interpretation. The evaluation should be read as review-routing evidence rather than a claim about fully automatic feedback quality. The public-corpus benchmark tests whether the router keeps constructed unsafe feedback out of automatic release; the teacher pilot checks whether the same signals align with human judgments on correctness, meaning preservation, readiness, and review necessity. The current policy is intentionally conservative: it sacrifices some automation rate in order to preserve all observed unsafe candidates in the review queue. This trade-off matches the target use case, where teacher time is limited but student-facing feedback should remain under human control.

7 Demo Scenario

The two-and-a-half-minute demo follows a teacher workflow: review one anonymized ESL draft, inspect graph paths, process a batch

Policy	Auto share	Auto acc.	Review share	Errors reviewed
Accept low; review med/high	0.329	1.000	0.671	1.000
Accept low/med; review high	0.336	1.000	0.664	1.000
Review all	0.000	–	1.000	1.000

Table 4: Pooled review-routing utility over the public learner-corpus benchmark. Errors reviewed is the fraction of observed incorrect or unsafe feedback candidates routed to the human-review queue.

CSV, compare AI feedback, handle a high-risk item in Teacher Queue, and export a report.

8 Scope and Design Choices

ConsensusScope is deliberately designed as a routing and observability layer, not as a new essay scorer or a replacement for the feedback generator. This choice reflects the deployment setting: schools and developers may change the underlying LLM provider, prompt, or feedback generator, but they still need a stable mechanism for deciding which feedback items can be shown to students. The unified schema separates generation from review, while the graph record keeps the decision auditable.

The system also separates deploy-time signals from offline diagnostic labels. At deploy time, the router can use agreement, issue type, target-span scope, parse status, evidence availability, meaning-change cues, and tone/specificity signals. It cannot know whether an item is a false consensus or a minority correct case unless such labels are produced by later evaluation. This separation is important for honest reporting: aggregate teacher ratings may calibrate the release policy offline, but per-item teacher labels and public correction labels are not read when routing a new student-facing item.

These design choices define the contribution boundary. ConsensusScope does not claim that a particular LLM produces high-quality educational feedback. Instead, it shows how feedback from one or more generators can be converted into auditable routing decisions before reaching students. This makes the system suitable for classroom-facing deployments where teachers want the efficiency of generative feedback but still need a defensible review process.

9 Availability and Reproducibility

ConsensusScope is released under the MIT License. The repository, live demo, video, and API documentation are available at https://github.com/yyting2006-ai/ConsensusScope, https://demo.consensusscope.cn/, https://youtu.be/vp4Kj48bv8Q, and https://api.consensusscope.cn/docs. The hosted demo uses a self-hosted Streamlit/FastAPI deployment. The repository includes startup instructions and scripts/run_public_gec_benchmark.py; raw corpora and per-item derived outputs are not committed.

10 Ethics and Limitations

ConsensusScope is a teacher-support tool, not an automatic essay scorer or teacher replacement. Users should upload only anonymized student writing. The benchmark validates graph-backed routing with public correction labels and constructed distractors; it does not measure learning gains, teacher satisfaction, time-on-task, or real LLM feedback quality. The 30-item two-teacher pilot is diagnostic, not a classroom study.

11 Conclusion

ConsensusScope demonstrates safety routing for generative AI English essay feedback. By defining feedback-level routing, extracting deploy-time risk signals, constructing item-level Feedback Safety Graphs, and sending uncertain or unsafe suggestions into a teacher queue, the system turns AI feedback from an opaque generated output into an auditable teacher-in-the-loop decision. This positioning addresses the central practical problem: reducing teachers' feedback burden without giving up control over what students receive.

References

Christopher Bryant, Mariano Felice, Øistein E. Andersen, and Ted Briscoe. 2019. The BEA-2019 shared task on grammatical error correction. In Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications, pages 52–75, Florence, Italy. Association for Computational Linguistics.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG evaluation using GPT-4 with better human alignment. arXiv preprint arXiv:2303.16634.

Nitin Madnani, Jill Burstein, Norbert Elliot, Beata Beigman Klebanov, Diane Napolitano, Slava Andreyev, and Maxwell Schwartz. 2018. Writing mentor: Self-regulated writing feedback for struggling writers. In Proceedings of the 27th International Conference on Computational Linguistics: System Demonstrations, pages 113–117, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Courtney Napoles, Keisuke Sakaguchi, and Joel Tetreault. 2017. JFLEG: A fluency corpus and benchmark for grammatical error correction. In Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers, pages 229–234, Valencia, Spain. Association for Computational Linguistics.

Hwee Tou Ng, Siew Mei Wu, Ted Briscoe, Christian Hadiwinoto, Raymond Hendy Susanto, and Christopher Bryant. 2014. The CoNLL-2014 shared task on grammatical error correction. In Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task, pages 1–14, Baltimore, Maryland. Association for Computational Linguistics.

Jim Ranalli. 2018. Automated written corrective feedback: how well can students make use of it? Computer Assisted Language Learning, 31(7):653–674.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations.

Helen Yannakoudakis, Ted Briscoe, and Ben Medlock. 2011. A new dataset and method for automatically grading ESOL texts. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pages 180–189, Portland, Oregon, USA. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In Advances in Neural Information Processing Systems.

A Supplementary Implementation Artifacts

This appendix is an artifact sheet for reproduction: pseudocode, calibrated weights, exported fields, and representative routed cases.

```
Input: feedback item f, context c, evidence e, agreement a
signals = extract_signals(f, c, e, a)
dims = map_to_graph_dimensions(signals)
s = bounded_risk_score(signals)
p = logistic(intercept + weights * features)
if parse_failure: route = needs_more_evidence
else if p >= cutoff or high_severity(dims): route = teacher_review
else: route = auto_accept
return route, p, s, dims, graph_nodes, graph_edges
```

Figure A.1: Routing pseudocode used by the demo implementation.

Calibration feature	Weight
Intercept	-0.582
Risk score	0.861
Local-edit signal	-0.467
Meaning-risk signal	1.138
Grounding-risk signal	0.653
Specificity-risk signal	-0.047
Agreement-risk signal	0.000
Evidence-gap signal	0.000

Table A.1: Pilot-calibrated logistic release policy. The cutoff is 0.300 and is stored in routing_calibration.json.

Output field	Stored value
risk_level	Risk label
recommended_action	Release route
Calibrated review probability	Review probability
Active graph dimensions	Active dimensions
safety_graph_path	Explanation path
safety_graph_nodes	JSON node records
safety_graph_edges	JSON edge records

Table A.2: Core fields exported for each routed feedback item.

Case	Pattern	Graph dimensions	Route	Diagnostic use
FB-002	Local phrase improvement	Local edit	Auto-accept	Teachers rate safe
FB-003	Reverses student thesis	Meaning preservation	Review	Blocks meaning change
FB-005	Wrong modal correction	Local edit; meaning preservation	Review	Finds unsafe local edit
FB-008	Unsupported exam-score claim	Meaning preservation; content grounding	Review	Blocks unsupported evidence

Table A.3: Representative pilot cases. Teacher ratings are offline diagnostics and are not per-item inputs during deploy-time routing.